Análise Comparativa de Eficiência e Acessos à Memória em Implementações da Estrutura de Dados Disjoint Set Union

João Victor Temponi Daltro de Castro[1]

[1]Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

joao.castro.1216657@sga.pucminas.br

Abstract. *This project evaluates the performance of three Disjoint Set Union (DSU) implementations: the Naive approach, Union by Rank, and Tarjan's path compression. These structures were integrated into Kruskal's algorithm to find the Minimum Spanning Tree in large-scale graphs. Results confirm that Tarjan's variant is the most efficient, achieving the lowest processing times. Data analysis shows that path compression drastically reduces repetitive memory access by shortening internal paths during search operations. The experiments empirically validate the underlying theory, demonstrating the structure's capability to perform in nearly constant time, even as data volume increases significantly.*

Resumo. *Este trabalho avalia o desempenho prático e teórico de três implementações da estrutura de dados Disjoint Set Union (DSU): a abordagem Ingênua, a heurística Union by Rank e a versão de Tarjan com compressão de caminho. As estruturas foram integradas ao algoritmo de Kruskal para identificar a Árvore Geradora Mínima em grafos de grande porte. Os resultados confirmam a superioridade da variante de Tarjan, que registrou os menores tempos de processamento. A análise demonstra que a compressão de caminho, ao encurtar rotas, minimiza leituras repetitivas na memória. Os testes validam a fundamentação teórica, comprovando que a otimização permite operações em tempo praticamente constante, mesmo com grandes volumes de dados.*

Bacharelado em Engenharia de *Software* - PUC Minas
Fundamentos de Projeto e Análise de Algoritmos (FPAA)

Professor da Disciplina: João Pedro Batisteli - joaobatisteli@pucminas.br

Belo Horizonte, 25 de Abril de 2026.

1. Introdução

Em diversos domıńios da computação, como o processamento de imagens, redes de computadores e bioinformática, surge a necessidade de gerenciar grupos de elementos que se fundem ao longo do tempo. O problema fundamental consiste em manter uma partição de um conjunto finito de n elementos em subconjuntos disjuntos, permitindo identificar rapidamente a qual grupo um elemento pertence e unir dois grupos distintos. Para isso, a estrutura de dados deve oferecer duas operações básicas. A operação `Find`, que identifica o representante do conjunto ao qual um elemento pertence, e a operação `Union`, que une dois conjuntos distintos.

Embora uma implementação ingênua utilizando listas ou árvores simples pareça suficiente, o desempenho degrada drasticamente conforme o número de operações aumenta. Esse efeito é crítico em algoritmos de grafos que repetem consultas de conectividade, como por exemplo o Kruskal, utilizado para encontrar a Árvore Geradora Mínima. A cada aresta considerada, é necessário verificar se suas extremidades já pertencem ao mesmo componente, não pertencendo, os componentes serão unidos. Em grafos grandes, pequenas diferenças no custo de `Find` e `Union` se acumulam e passam a dominar o tempo de execução.

Este trabalho analisa o efeito prático de heurísticas clássicas do DSU. Em particular, são avaliadas as heurísticas *Union by Rank* e *Path Compression*, discutidas por Cormen *et al.* [1] (Cap. 21–21.4). Essas técnicas reduzem o custo amortizado das operações e tornam o DSU adequado para instâncias em larga escala.

Quatro variantes são comparadas para elaboração das análises. A primeira é uma implementação ingênua, sem heurísticas. A segunda aplica *Union by Rank*. A terceira combina *Union by Rank* e *Path Compression*, correspondendo ao *Full Tarjan*. A quarta é uma variação otimizada do *Full Tarjan*, na qual o `Find` foi ajustado para reduzir acessos redundantes ao vetor. A comparação utiliza o tempo de execução do laço principal do Kruskal e a contagem de acessos aos vetores internos do DSU, de modo a isolar o impacto das heurísticas.

Como resultado, o trabalho apresenta uma implementação modular em C++ que permite comparar as quatro variantes sob o mesmo cenário, avalia uma no `Find` e discute em quais escalas as versões se destacam ou se tornam inviáveis.

2. Metodologia

Esta seção descreve o ambiente de execução, as decisões de implementação, a geração das instâncias e o protocolo de medição adotado para comparar as variantes do DSU.

2.1. Ambiente de Execução e Ferramentas

Os experimentos foram conduzidos em uma estação de trabalho equipada com o sistema operacional Pop!_OS 22.04 LTS, processador AMD Ryzen 9 5900XT (16 núcleos) e 48 GB de memória RAM.

Conforme as diretrizes do projeto, a solução principal foi implementada em C++. A escolha se justifica pelo desempenho necessário em instâncias de grande porte e pelo controle mais direto sobre os recursos da linguagem. O código foi compilado com `g++` (versão 11.4.0) e com a opção `-O3` para melhor desempenho.

A linguagem Python (versão 3.10.12) foi empregada de forma auxiliar na geração das instâncias de entrada (grafos) e em protótipos de verificação em pequena escala. Os resultados reportados neste trabalho, entretanto, referem-se apenas às medições executadas na aplicação com C++.

2.2. Arquitetura da Implementação

A implementação em C++ seguiu o Princípio da Responsabilidade Única (SRP) e os conceitos de Orientação a Objetos. O código foi organizado separando as declarações

das funções, em arquivos de cabeçalho (`.hpp`), de suas respectivas lógicas, em arquivos fonte (`.cpp`).

Para padronizar o sistema, foi criada uma classe base que encapsula os vetores internos `pai` e `altura` e oferece métodos instrumentados de leitura e escrita. Esses métodos incrementam um contador a cada acesso, o que permite medir o custo de memória de forma consistente entre as variantes. As quatro variantes avaliadas herdam e implementam a mesma interface.

A organização modular simplificou a etapa de testes, onde uma única função avaliadora executa o laço do Kruskal para todas as variantes, alterando apenas o DSU instanciado. Essa padronização também permitiu desativar a variante ingênua em escalas de n onde seu tempo se torna impraticável.

Além disso, a separação do código oferece segurança para aplicar melhorias localizadas. Em particular, a variante *Full Tarjan* Otimizada altera apenas o método `Find` para evitar uma leitura redundante do vetor `pai` ao final da recursão, reduzindo acessos contabilizados sem modificar a lógica de execução do Kruskal.

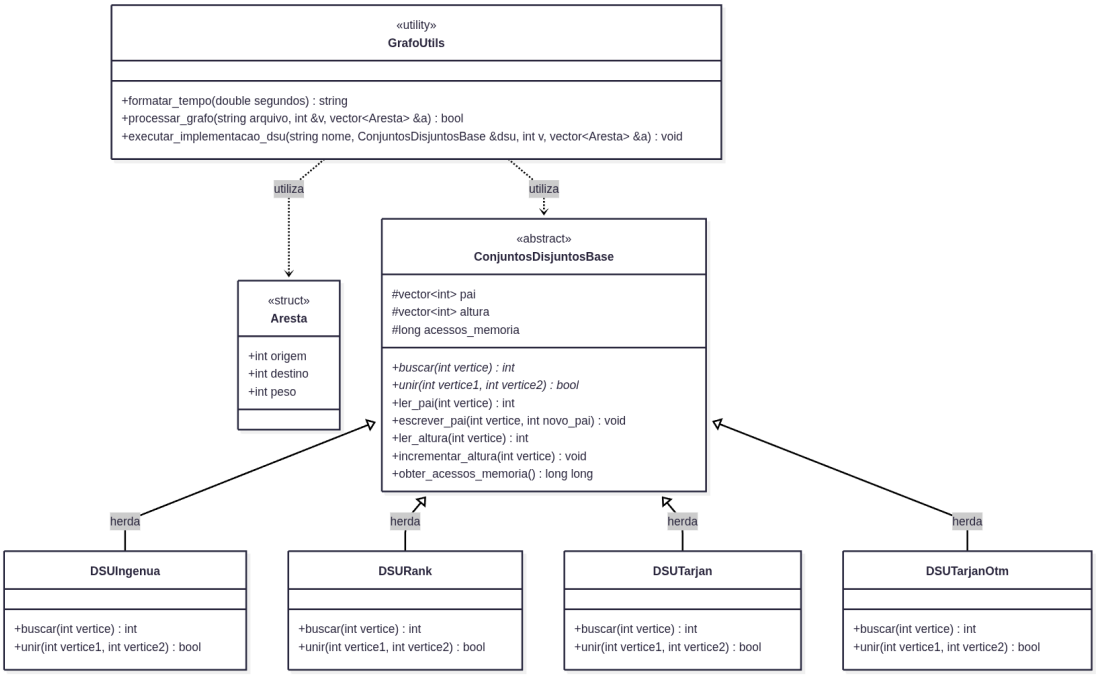

Figura 1. Diagrama de classe simplificado, implementação em c++

2.3. Geração de Dados e Cenários de Teste

A validação empírica da complexidade assintótica de $O(\alpha(n))$ demanda instâncias de entrada em larga escala. Inicialmente, cogitou-se o uso de modelos de inteligência artificial generativa para a confecção dos grafos conexos. No entanto, essas ferramentas apresentaram baixa capacidade de manter a formatação estruturada e falharam em garantir a conectividade do grafo para instâncias contendo mais de 500 arestas.

Para viabilizar grandes instâncias de entrada, foi desenvolvido um *script* em Python para gerar grafos no formato `origem;destino;peso`. O objetivo do gera-

dor é produzir entradas conexas e parametrizadas por número de vértices V e arestas E. O processo começa pela criação de uma árvore base com $V - 1$ arestas, o que garante conectividade. Para cada vértice i entre 1 e $V - 1$, é escolhido um destino j entre 0 e $i - 1$. Em seguida, grava-se a aresta entre i e j, com o par normalizado para que a origem seja o menor identificador. Os pesos são números inteiros aleatórios no intervalo de 1 a 100. Essa etapa é vetorizada com NumPy, o que reduz o tempo de geração.

Após a árvore base, o gerador cria as arestas restantes em lotes, com tamanho definido pelo parâmetro `LOTE`. Essa estratégia evita o uso de estruturas como *set*, que exigem memória proporcional a E e se tornam inviáveis em escalas muito grandes. Em cada lote, são sorteados pares (u, v) no intervalo de 0 a $V - 1$ e são descartados laços internos. As arestas válidas são normalizadas para manter a mesma convenção de ordenação e então são gravadas no arquivo. Como não há armazenamento global das arestas, o gerador não exige unicidade estrita. Duplicatas não afetam a correção do Kruskal, mas podem aumentar o tamanho do arquivo e o tempo total de execução.

2.4. Desafios e Limitações Práticas

A execução do trabalho envolveu desafios para experimentos em grande escala. Do ponto de vista de referências, a principal base teórica utilizada foi o tratamento apresentado por Cormen *et al.* [1] (Cap. 21–21.4). Fora desse material, poucas fontes de fácil acesso que discutam o tema com o nível de detalhe necessário foram encontradas.

Outro desafio ocorreu durante a geração das instâncias de entrada. Para atingir as escalas necessárias de n, foi necessário construir novo e otimizado *script* em Python. A versão inicial utilizava um *set* para evitar duplicatas, mas essa abordagem exigia memória proporcional ao número de arestas. Por esse motivo, o gerador foi reescrito para gerar e gravar as arestas por lotes, o que permitiu produzir arquivos maiores e manter a execução viável.

```python
def gerar_grafo(qtd_vertices, qtd_arestas, nome_arquivo="grafo.txt"):
    if qtd_arestas < qtd_vertices - 1:
        print("Erro: O grafo precisa de V-1 arestas.")
        return

    rng = np.random.default_rng()

    with open(f"{CAMINHO}{nome_arquivo}", 'w') as f:
        origens = np.arange(1, qtd_vertices, dtype=np.int32)
        destinos = (rng.random(qtd_vertices - 1) * origens).astype(np.int32)
        pesos = rng.integers(1, 101, size=qtd_vertices - 1, dtype=np.int8)

        u = np.minimum(origens, destinos)
        v = np.maximum(origens, destinos)
        arvore = np.column_stack((u, v, pesos))
        np.savetxt(f, arvore, fmt='%d;%d;%d')

        del origens, destinos, u, v, arvore
        arestas_restantes = qtd_arestas - (qtd_vertices - 1)
        geradas = 0

        while geradas < arestas_restantes:
            lote_atual = min(LOTE, arestas_restantes - geradas)
            orig = rng.integers(0, qtd_vertices, size=lote_atual, dtype=np.int32)
            dest = rng.integers(0, qtd_vertices, size=lote_atual, dtype=np.int32)

            validas = orig != dest
            orig = orig[validas]
            dest = dest[validas]

            if len(orig) == 0:
                continue

            pesos = rng.integers(1, 101, size=len(orig), dtype=np.int8)
            u = np.minimum(orig, dest)
            v = np.maximum(orig, dest)

            chunk = np.column_stack((u, v, pesos))
            np.savetxt(f, chunk, fmt='%d;%d;%d')
            geradas += len(orig)

    print(f"{nome_arquivo} com {qtd_vertices} vértices e {qtd_arestas} arestas.")

gerar_grafo(VERTICES, ARESTAS, "grafo.txt")
```

Figura 2. *Script* desenvolvido em Python para a geração dos grafos de entrada e validação de conectividade.

As instâncias também impuseram limites de armazenamento e de entrada e saída. Em particular, grafos com um bilhão de arestas resultaram em arquivos com mais de 20 gigabytes, o que restringiu a execução de testes ainda maiores. Por fim, como o desenvolvimento deveria ser implementado obrigatoriamente em C, C++ ou Java, optou-se por um protótipo inicial em Python apenas para validar a lógica e familiarizar com o desenvolvimento das variantes. A implementação final foi aprimorada e reescrita de forma **manual**, organizada e modular em C++, com foco em desempenho para grandes volumes de dados.

2.5. Procedimentos de Medição

O desempenho das variantes foi medido com base em dois critérios: tempo de processamento e número de acessos à memória interna do DSU.

O tempo foi contabilizado com a biblioteca `<chrono>` em torno do laço principal do Kruskal, isto é, sobre as operações de seleção de arestas e chamadas ao DSU até completar $V - 1$ uniões. A ordenação das arestas por peso é realizada uma única vez

antes das medições, pois o objetivo do experimento é analisar isoladamente o impacto das heurísticas do DSU.

O custo de memória foi aferido incrementando um contador a cada operação de leitura ou escrita nos vetores internos `pai` e `altura`. Dessa forma, a métrica reflete o trabalho efetivo do DSU, ao invés de tentativas indiretas de medir consumo de RAM do processo.

Em escalas onde a abordagem ingênua apresentou tempo de execução inviável, acima de 50 minutos, o monitoramento via sistema operacionalfoi utilizado para registrar o tempo decorrido até a suspensão manual do processo (Figura 3).

Figura 3. Monitoramento e suspensão do processo da variante Ingênua por tempo viável excedido.

3. Fundamentação Teórica

A estrutura de dados para Conjuntos Disjuntos, conhecida como *Disjoint Set Union* (DSU) ou *Union-Find*, mantém uma partição de um conjunto de n elementos em subconjuntos disjuntos. A estrutura suporta duas operações: `Find(x)`, que retorna o representante (raiz) do conjunto que contém x, e `Union(x,y)`, que une os conjuntos de x e y caso sejam distintos. A implementação mais comum representa cada conjunto como uma árvore enraizada, em que cada nó aponta para seu pai e a raiz aponta para si mesma.

Em uma implementação ingênua, sem heurísticas de balanceamento, uniões sucessivas podem degenerar a árvore em uma cadeia (estrutura semelhante a lista), fazendo com que `Find` percorra quase todos os nós até a raiz. Nesse cenário, o custo pode chegar a $O(n)$ no pior caso, tornando a estrutura impraticável em aplicações com muitas operações.

Para evitar essa degradação, aplica-se a heurística de união por posto (*Union by Rank*). A estratégia consiste em sempre anexar a árvore com a menor profundidade à raiz da árvore mais profunda durante a mesclagem. Conforme discutido em Cormen *et al.* [1] (Cap. 21–21.4), essa técnica assegura que a altura máxima de qualquer árvore resultante seja limitada por $\log_2 n$. Dessa forma, o custo computacional no pior caso para cada operação isolada é reduzido para $O(\log n)$.

A heurística de *Union by Rank* pode ser vista como um balanceamento. Ao unir, compara o posto das raízes e faz a raiz de menor posto apontar para a raiz de maior posto. Quando os postos são iguais, escolhe-se uma raiz como pai e incrementa-se seu posto. Essa regra limita o crescimento da altura das árvores ao longo das uniões.

A segunda otimização, denominada compressão de caminho (*Path Compression*), atua durante o `Find`. Ao percorrer a árvore em direção à raiz, o algoritmo faz com que cada nó visitado passe a apontar diretamente para o representante final. Essa técnica achata as árvores gradualmente, reduzindo o custo de buscas futuras no mesmo conjunto.

A análise da aplicação simultânea da união por posto e da compressão de caminho demonstra a eficiência da variante de Tarjan. Segundo Cormen *et al.* [1] (Cap. 21–21.4), a combinação dessas duas técnicas reduz o tempo amortizado por operação para a complexidade de:

$$O(\alpha(n))$$

Onde $\alpha(n)$ representa a função inversa da função de Ackermann. A função de Ackermann apresenta uma taxa de crescimento acentuada, o que confere à sua inversa, $\alpha(n)$, um crescimento assintótico excepcionalmente lento.

Em termos práticos, para valrores de n usados em aplicações, $\alpha(n)$ é pequeno, em geral menor ou igual a 4. Isso significa que o custo amortizado por operação é próximo de constante. Deve-se notar que a analise é amortizada. Algumas operações podem custar mais, mas a média ao longo de uma sequência longa permanece baixa.

No contexto do algoritmo de Kruskal, o DSU é invocado repetidamente para testar se duas extremidades já pertencem ao mesmo componente e para unir componentes quando uma aresta é escolhida. Assim, qualquer redução no custo amortizado de `Find`/`Union` impacta diretamente o tempo total do laço principal do Kruskal, especialmente em instâncias com grande número de arestas.

4. Discussão e Resultados

Esta seção apresenta e discute os resultados experimentais obtidos para diferentes implementações de *Disjoint Set Union* (DSU), variando a escala do problema por faixas de vértices–arestas (V–E). As medições foram realizadas durante a execução do laço principal do algoritmo de Kruskal, seleção de arestas com chamadas a `Union`/`Find`), **excluindo** o custo de ordenação das arestas, que é executado uma única vez antes dos testes. Foram observadas duas métricas, o tempo de execução (ms) e a quantidade de acessos à memória interna do DSU. As Tabelas 1 e 2 consolidam os valores medidos, as Figuras 4 e 5 apresentam a evolução em escala logarítmica, e as Figuras 6–9 detalham comparações diretas entre as variantes de Tarjan e o *Union by Rank*.

Tabela 1. Tempo de execução (ms) por escala V–E.

Implementação	10–100	100–1000	1000–10mil	10mil–100mil	100mil–1M	1M–10M	10M–100M	100M–1B
Ingênua	0,000751	0,038952	4,55	596,16	58711,5	–	–	–
Union by Rank	0,000731	0,019126	0,225282	3,21	36,12	570,88	15940,1	388400
Full Tarjan	0,000861	0,018415	0,204443	2,57	28,19	416,4	10957,6	237330
Full Tarjan – Otimizado	0,000791	0,014988	0,148929	2	23,02	348,72	10114,8	225460

Tabela 2. Quantidade de acessos à memória por escala V–E.

Implementação	10–100	100–1000	1000–10mil	10mil–100mil	100mil–1M	1M–10M	10M–100M	100M–1B
Ingênua	49	8279	1497765	208108913	19567649261	–	–	–
Union by Rank	72	2818	38733	558652	5465580	67363136	804211154	8860014757
Full Tarjan	102	3348	42592	583643	5561228	67690747	795897331	8694893222
Full Tarjan – Otimizado	72	2128	26773	362440	3459676	41842996	489825906	5339825015

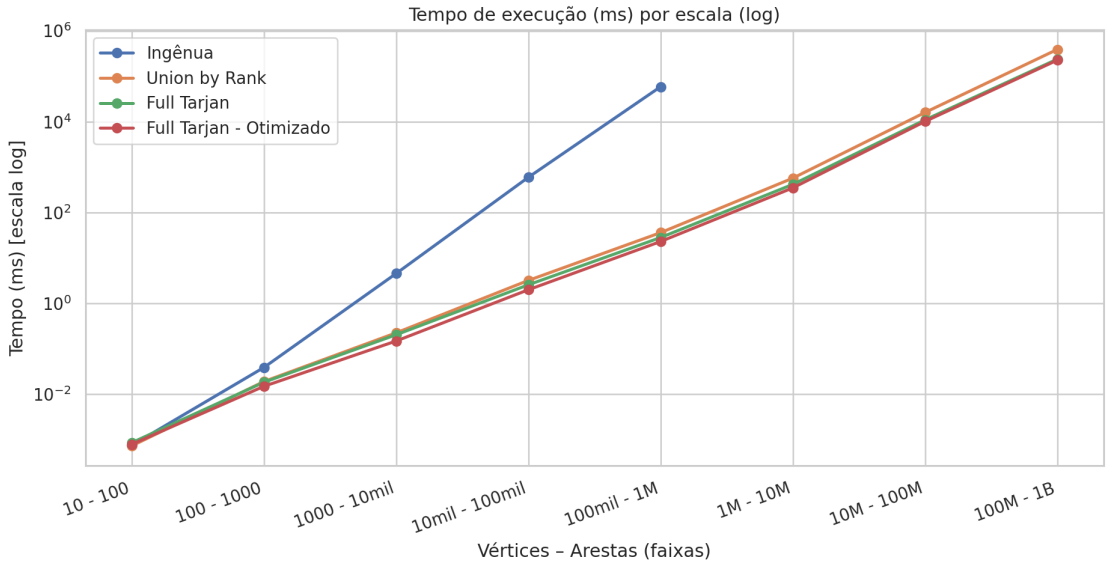

Figura 4. Tempo de execução por escala V–E (eixo y em escala logarítmica).

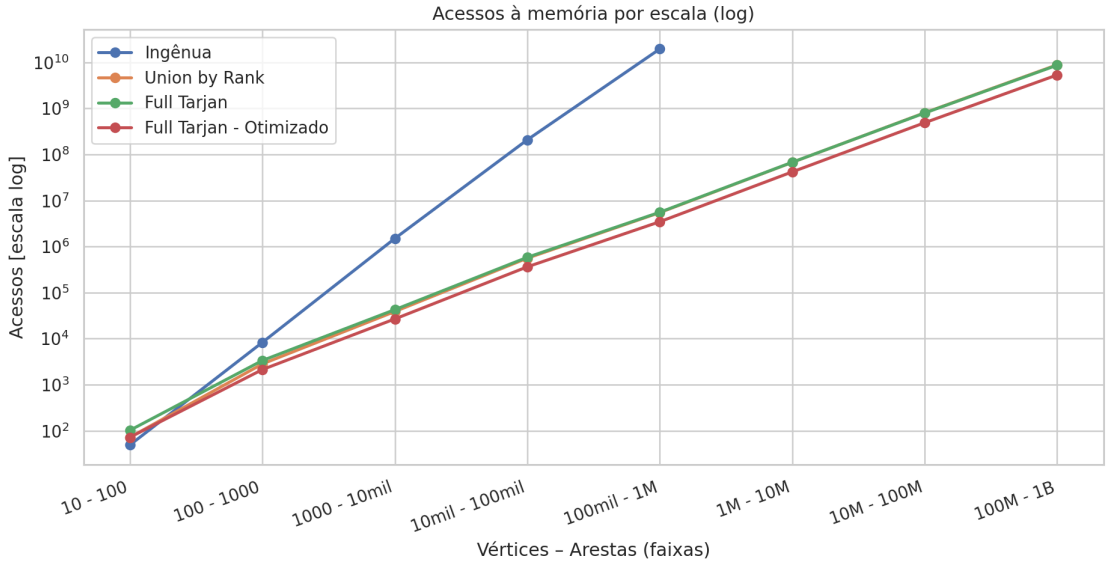

Figura 5. Acessos à memória por escala V–E (eixo y em escala logarítmica).

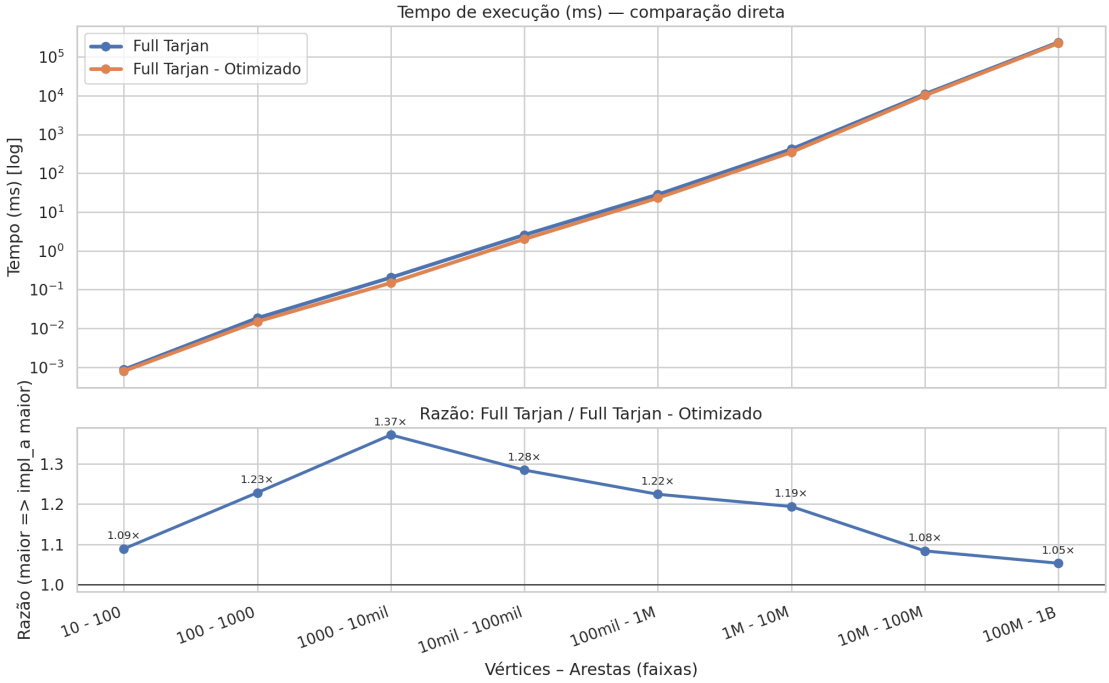

Figura 6. Comparação direta de tempo: Full Tarjan vs Full Tarjan–Otimizado, com a razão (speedup) por faixa.

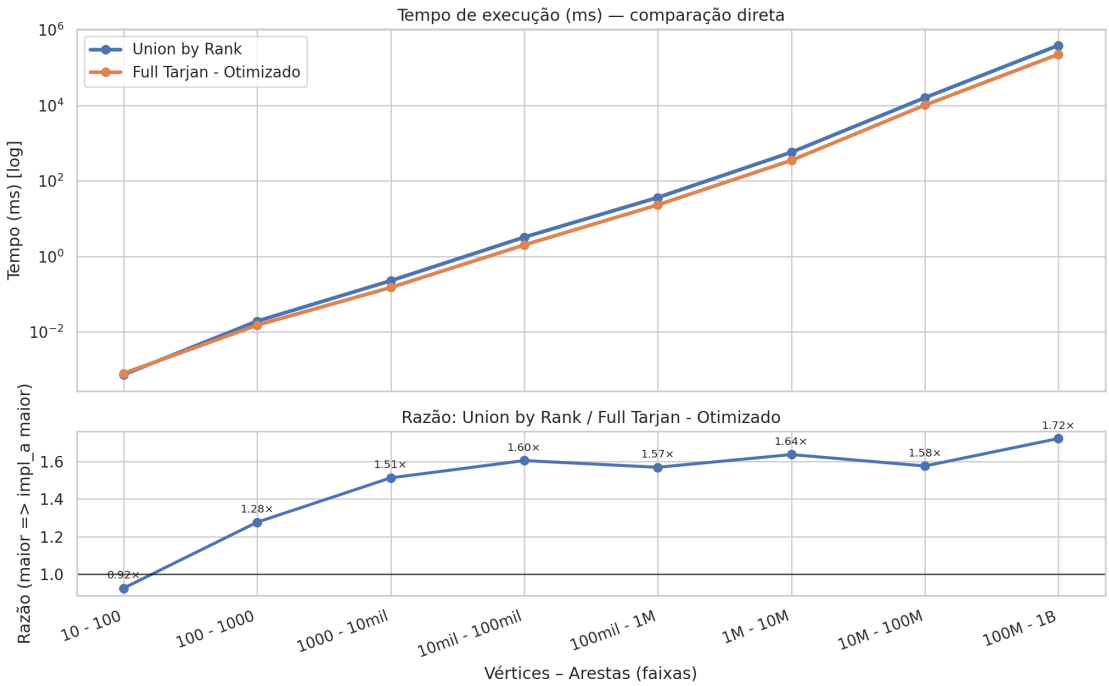

Figura 7. Comparação direta de tempo: Union by Rank vs Full Tarjan–Otimizado, com a razão (speedup) por faixa.

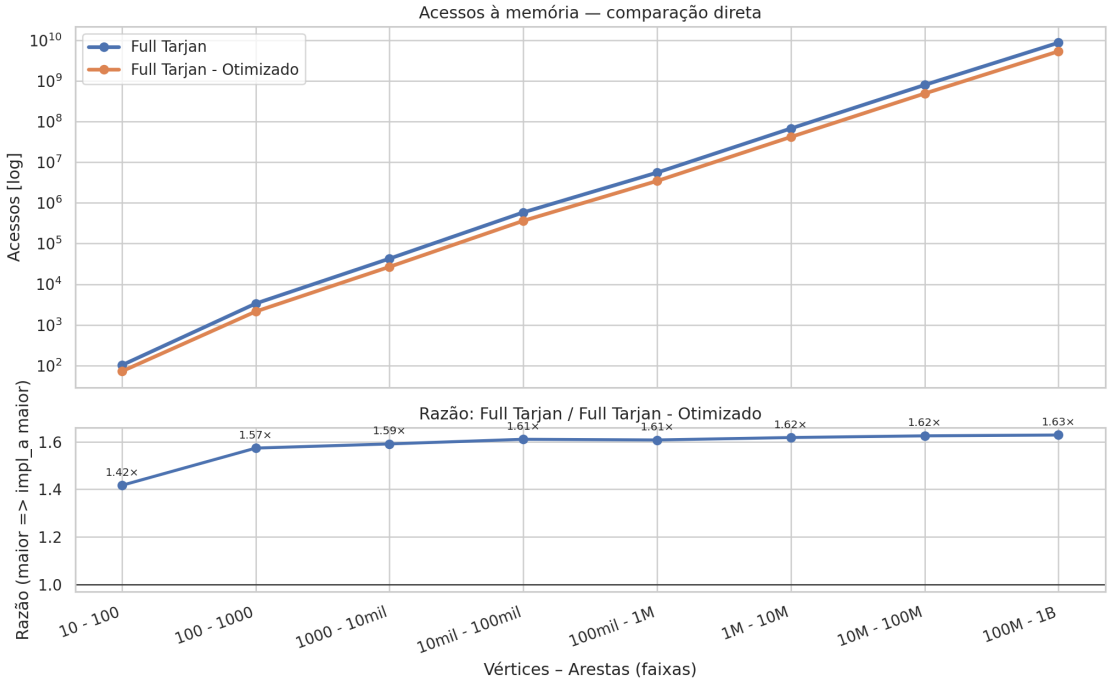

Figura 8. Comparação direta de acessos à memória: Full Tarjan vs Full Tarjan–Otimizado, com a razão por faixa.

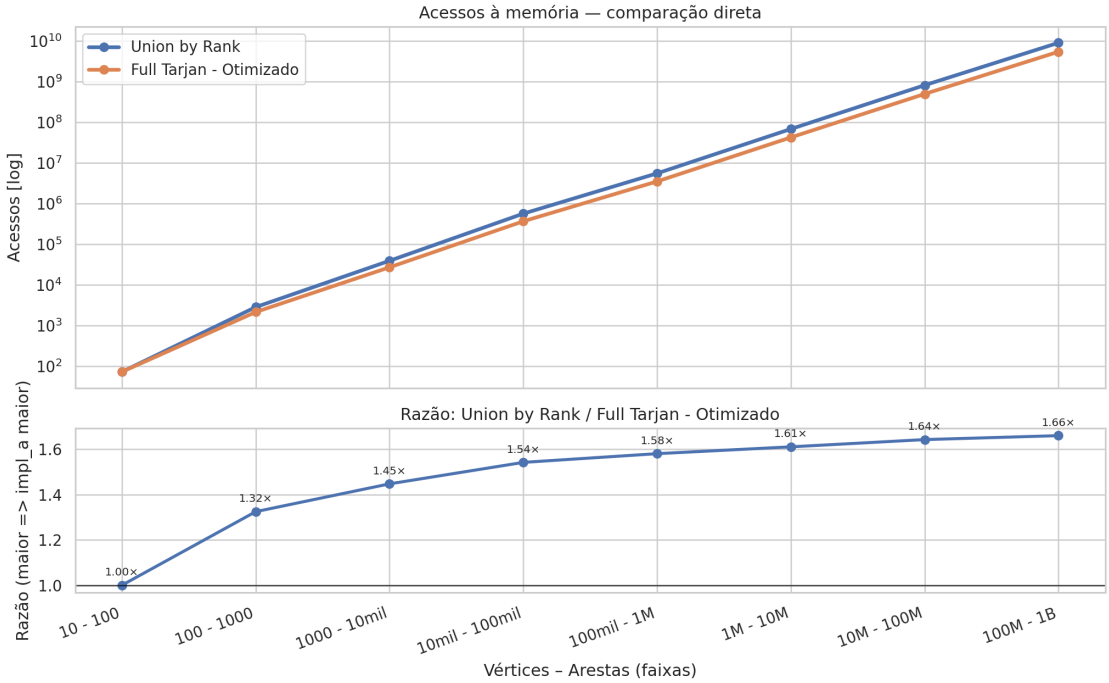

Figura 9. Comparação direta de acessos à memória: Union by Rank vs Full Tarjan–Otimizado, com a razão por faixa.

4.1. Tendências gerais e escalabilidade.

As Figuras 4 e 5 evidenciam crescimento das métricas com o aumento da escala V–E, justificando o uso de eixos em log para melhor visualização. Como o laço do Kruskal executa um volume proporcional de operações sobre o DSU, até selecionar $V - 1$ arestas para a MST, o comportamento observado no experimento é sensível ao custo computacional de `Find`/`Union`. A implementação ingênua degrada rapidamente: o tempo sai de 4,55 ms (1000–10mil) para 596,16 ms (10mil–100mil) e alcança 58711,5 ms em 100mil–1M (Tabela 1). Na mesma progressão, seus acessos saltam para 19,6 bilhões em 100mil–1M (Tabela 2). Além do custo por operação crescer, a ausência de compressão de caminho e balanceamento torna o comportamento inviável em escalas maiores, impossibilitando a execução para entradas a partir de 1M–10M, configurando um limite prático de aplicabilidade.

4.2. Melhor versão para cada cenário.

Em escalas muito pequenas (10–100), as diferenças de tempo entre implementações otimizadas são reduzidas e podem ser ofuscadas por custos fixos de execução e ruído de medição. Por exemplo, *Union by Rank* (0,000731 ms) é ligeiramente menor do que Full Tarjan–Otimizado (0,000791 ms), apesar de ambos estarem na mesma ordem de grandeza (Tabela 1). A partir de 100–1000, entretanto, as versões com compressão de caminho tornam-se consistentemente superiores: Full Tarjan–Otimizado passa a ser o menor tempo em todas as faixas onde há dados completos, indicando que o benefício da compressão supera seus custos constantes quando a instância cresce.

4.3. Full Tarjan vs Full Tarjan–Otimizado.

As Figuras 6 e 8 mostram que a versão otimizada reduz simultaneamente tempo e acessos em todas as faixas. No código, essa diferença decorre de uma otimização pontual no `Find`. A variante otimizada retorna o representante calculado de forma direta, evitando uma leitura adicional do vetor de "pais" ao final da recursão, eliminando um acesso redundante por chamada em comparação com a forma implementada inicialmente. Analisando o tempo, a melhora do otimizado sobre o Full Tarjan varia aproximadamente de 1,08× a 1,37× ao longo das escalas, (0,204443 ms → 0,148929 ms em 1000–10mil; e 10957,6 ms → 10114,8 ms em 10M–100M).

Em acessos, o ganho é mais estável e evidente, o Full Tarjan realiza cerca de 1,42× a 1,62× mais acessos que o otimizado (42592 → 26773 em 1000–10mil; e 795897331 → 489825906 em 10M–100M). O fato de as razões de acessos serem mais consistentes que as de tempo sugere que o tempo do experimento é, em grande parte, controlado pelo volume de leituras e escritas do DSU, enquanto variações residuais podem refletir efeitos de cache e do escalonamento do sistema.

4.4. Union by Rank vs Full Tarjan–Otimizado.

Nas Figuras 7 e 9, o que o Tarjan otimizado também supera o *Union by Rank* em praticamente todas as escalas relevantes, sobretudo quando o tamanho do problema cresce. Analizando o tempo, a razão *Union by Rank*/*Tarjan Otimizado* permanece próxima de 1,5–1,6× nas maiores faixas (por exemplo, 570,88 ms → 348,72 ms em 1M–10M, e 15940,1 ms → 10114,8 ms em 10M–100M).

Em acessos à memória, o padrão acompanha o do tempo: o *Union by Rank* chega a realizar cerca de 1,61–1,64× mais acessos que o Tarjan otimizado nas faixas maiores, por exemplo (67363136 → 41842996 em 1M–10M; e 804211154 → 489825906 em 10M–100M). Na menor faixa (10–100), ambos apresentam o mesmo número de acessos (72), reforçando que em tamanhos pequenos o custo fixo e o ruído tendem a dominar, enquanto a superioridade do Tarjan otimizado emerge com clareza em escalas maiores.

Na maior instância avaliada (100M–1B), o comportamento se mantém: *Union by Rank* demanda 388400 ms (6 min 28,40 s), enquanto o Tarjan otimizado executa em 225460 ms (3 min 45,46 s), correspondendo a um ganho de aproximadamente 1,72× em tempo. Em termos de acessos, *Union by Rank* realiza 8860014757 acessos contra 5339825015 do Tarjan otimizado (cerca de 1,66× a mais), reforçando a associação entre redução de acessos nos vetores internos do DSU e redução do tempo total do laço do Kruskal.

4.5. Análise geral das variantes.

Os resultados indicam uma relação direta entre o tempo e a quantidade de acessos à memória. Nas escalas maiores, as variantes com menos leituras e escritas tendem a apresentar menor tempo total. Este efeito é esperado, pois a métrica de acessos contabiliza operações nos vetores `pai` e `altura`, que concentram o custo do DSU. Como o restante do laço do Kruskal é o mesmo para todas as variantes, a redução de acessos tende a se refletir em redução do tempo medido. Dessa forma, a degradação mais forte ocorre quando a implementação aumenta o número de acessos, como na versão ingênua, enquanto o melhor desempenho aparece quando esse volume é reduzido, como no Tarjan otimizado.

Conforme análise, a implementação ingênua se mostrou aceitável apenas em instâncias pequenas, degradando rapidamente. O *Union by Rank* oferece boa escalabilidade, porém é superada pelas implementações baseadas em Tarjan Full. O Full Tarjan–Otimizado é a opção mais robusta, pois mantém o menor tempo e o menor número de acessos à memória ao longo das maiores faixas avaliadas, destacando-se como a melhor escolha para instâncias de grande escala.

5. Conclusão

Este trabalho investigou o efeito prático de heurísticas clássicas em *Disjoint Set Union* no contexto do algoritmo de Kruskal. Foram comparadas quatro variantes. A primeira é a versão ingênua. A segunda aplica *Union by Rank*. A terceira combina *Union by Rank* e *Path Compression*, que corresponde ao *Full Tarjan*. A quarta é uma variação otimizada do *Full Tarjan*. A expectativa, conforme Cormen *et al.* [1] (Cap. 21–21.4), é que a combinação de balanceamento e compressão reduza o custo amortizado das operações e amplie a viabilidade em instâncias de grande escala.

Os resultados confirmam essa expectativa. A variante ingênua degrada rapidamente à medida que a escala cresce, enquanto as variantes com heurísticas mantêm desempenho mais estável. As variantes baseadas em Tarjan apresentaram os menores tempos e também o menor número de acessos aos vetores internos do DSU. Esse comportamento reforça que o custo do experimento é influenciado pelo volume de leituras e escritas em `pai` e `altura`. Na maior instância avaliada, com 100 milhões de vértices e 1 bilhão de arestas, o *Full Tarjan* otimizado manteve vantagem clara sobre *Union by Rank*, com menor tempo e menos acessos.

Referências

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2012). **Algoritmos: teoria e prática**. 3ª edição. Elsevier.